

- **Course Title: Introduction to Database Systems**

- **Course Description:**
- **This course provides a comprehensive introduction to the design, use, and internal workings of database management systems.**
- **learn how to model data, write complex SQL queries, design relational databases, and understand the core system architecture that makes databases efficient, reliable, and scalable (including storage, indexing, and transaction management).**

- **Prerequisites: * Introductory Programming (Python, Java, or C++)**

- **Data Structures and Algorithms**
- **Basic Discrete Mathematics**



Part 1: Introduction to Databases

Contents



- Example of a Database (UNIVERSITY)
- Main Characteristics of the Database Approach
- Types of Database Users
- Advantages of Using the Database Approach
- Historical Development of Database Technology
- Extending Database Capabilities
- When Not to Use Databases

Recap: Basic Definitions



- **Database:**
  - A collection of related data.
- **Data:**
  - Known facts that can be recorded and have an implicit meaning.
- **Mini-world:**
  - Some part of the real world about which data is stored in a database.
    - E.g., student grades and transcripts at a university.
- **Database Management System (DBMS):**
  - A software package/system to facilitate the creation and maintenance of a computerized database.
- **Database System:**
  - The DBMS software together with the database itself. Sometimes, the applications are also included.

Simplified DB system environment

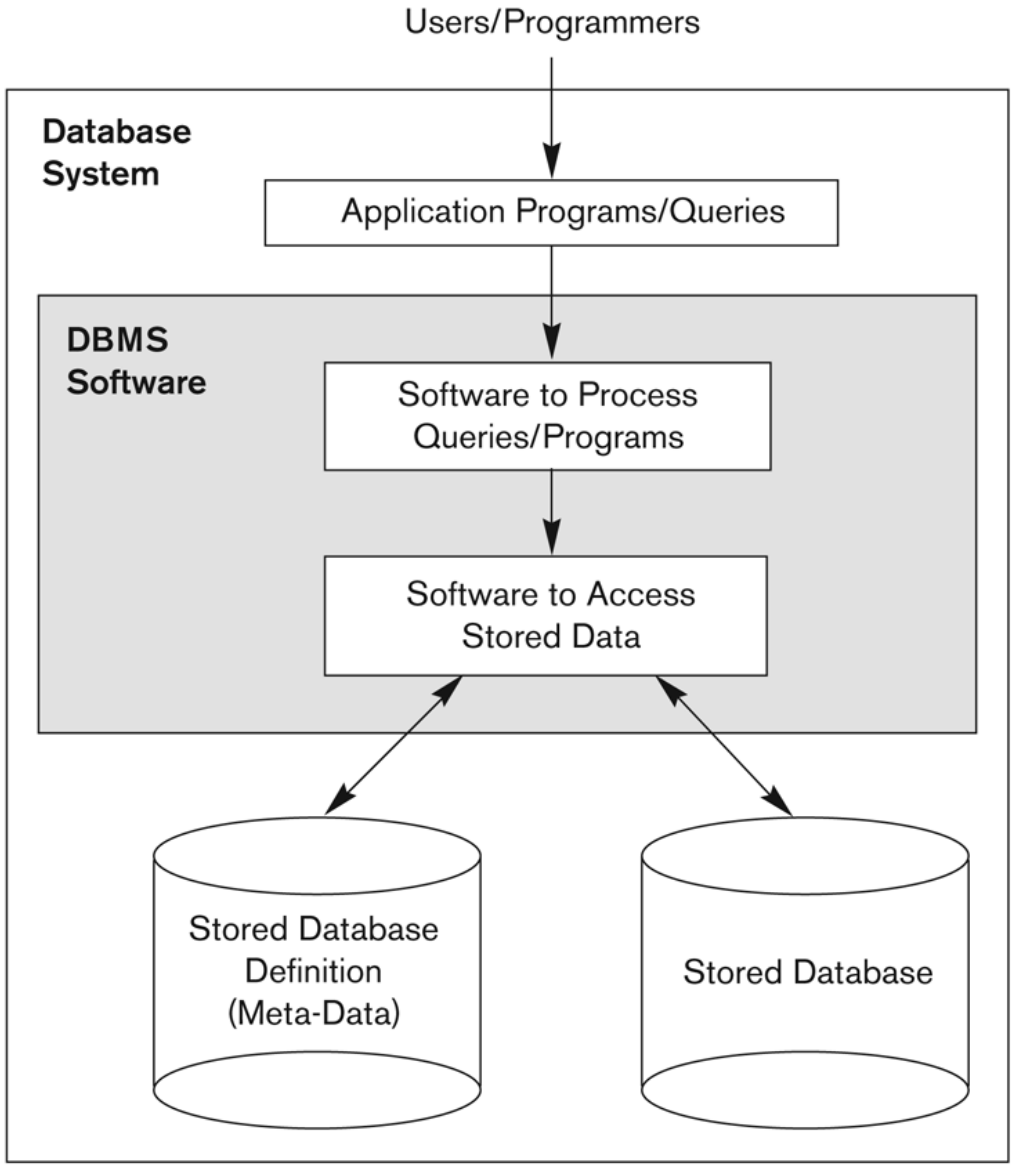


Figure 1.1
A simplified database system environment.

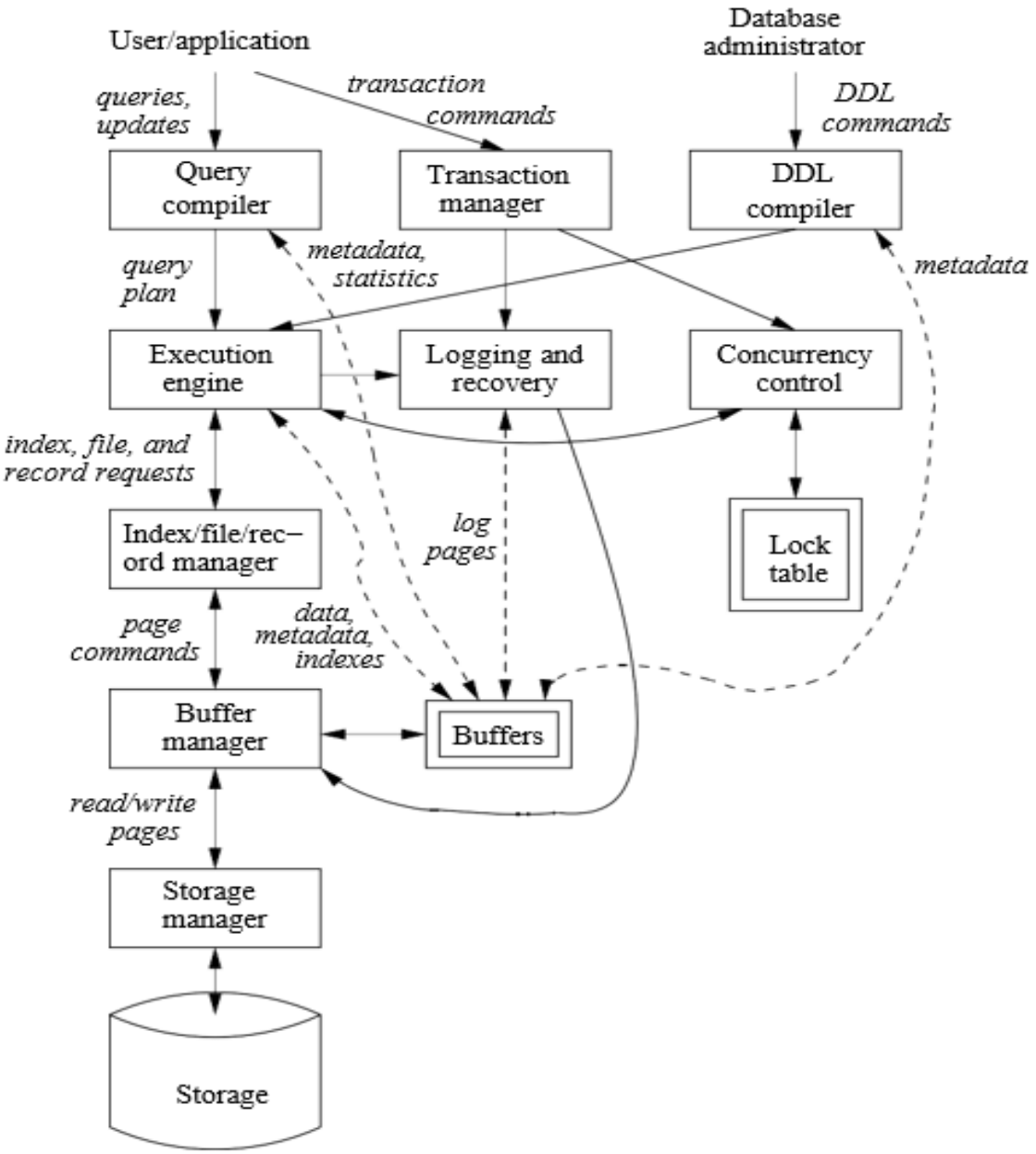


Figure 1: Database management system components

Example of a Database
(with a Conceptual Data Model)



- **Mini-world for the example:**
  - Part of a UNIVERSITY environment.
- **Some mini-world *entities*:**
  - STUDENTs
  - COURSEs
  - SECTIONs (of COURSEs)
  - (academic) DEPARTMENTs
  - INSTRUCTORs

Example of a Database
(with a Conceptual Data Model)



- **Some mini-world *relationships*:**
  - SECTIONs *are of specific* COURSEs
  - STUDENTs *take* SECTIONs
  - COURSEs *have prerequisite* COURSEs
  - INSTRUCTORs *teach* SECTIONs
  - COURSEs *are offered by* DEPARTMENTs
  - STUDENTs *major in* DEPARTMENTs

- **Note: The above entities and relationships are typically expressed in a conceptual data model, such as the ENTITY-RELATIONSHIP data model (we will learn it later)**

Example of a simple database



STUDENT

Name	Student_number	Class	Major
Smith	17	1	CS
Brown	8	2	CS

COURSE

Course_name	Course_number	Credit_hours	Department
Intro to Computer Science	CS1310	4	CS
Data Structures	CS3320	4	CS
Discrete Mathematics	MATH2410	3	MATH
Database	CS3380	3	CS

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
85	MATH2410	Fall	04	King
92	CS1310	Fall	04	Anderson
102	CS3320	Spring	05	Knuth
112	MATH2410	Fall	05	Chang
119	CS1310	Fall	05	Anderson
135	CS3380	Fall	05	Stone

GRADE_REPORT

Student_number	Section_identifier	Grade
17	112	B
17	119	C
8	85	A
8	92	A
8	102	B
8	135	A

PREREQUISITE

Course_number	Prerequisite_number
CS3380	CS3320
CS3380	MATH2410
CS3320	CS1310

Figure 1.2
A database that stores student and course information.

Database Approach vs. File Processing



- **In traditional file-processing:**
  - Ad-hoc implementations
  - Wasted storage space
  - Redundant efforts to maintain common up-to-date data
- **In the database approach:**
  - A single repository
  - Unified and repeated access

Main Characteristics of the Database Approach



- **Self-describing nature of a database system:**
  - A DBMS **catalog** stores the description of a particular database (e.g. data structures, types and formats, and constraints)
  - The description is called **meta-data***.
  - This allows the DBMS software to work with different database applications.

---

* Some newer systems such as a few NOSQL systems need no meta-data: they store the data definition within its structure making it self describing

Main Characteristics of the Database Approach



- **Insulation between programs and data:**
  - Called **program-data independence**.
  - Allows changing data structures and storage organization without having to change the DBMS access programs.

An example of file-processing software: Finding IP Address of Internet Domain Name (see next slide)

*the file structure of the record is already coded within each program that accesses the file.
By contrast, whenever a request is made to access the file, the DBMS refers to the catalog to determine the structure



```
#include <string.h>

#define    DOMAINNAMELENGTH  256
#define    DDNNAMELENGTH      30
#define    MAXHOSTS            5

typedef  struct  {
     char    domainname[DOMAINNAMELENGTH];
     char    ddnIP[DDNNAMELENGTH];
}  tHOST;


tHOST    hosttable[MAXHOSTS];
static   int next = 0;       // The index of

bool FindName(char *name)
{
     int k;
     for (k = 0; k < next; k++){
          if (strcmp(hosttable[k].domainnam
               return true;
     }
     return false;
}
```

Example of a simplified database catalog



RELATIONS

Relation_name	No_of_columns
STUDENT	4
COURSE	4
SECTION	5
GRADE_REPORT	3
PREREQUISITE	2

COLUMNS

Column_name	Data_type	Belongs_to_relation
Name	Character (30)	STUDENT
Student_number	Character (4)	STUDENT
Class	Integer (1)	STUDENT
Major	Major_type	STUDENT
Course_name	Character (10)	COURSE
Course_number	XXXXNNNN	COURSE
….	….	…..
….	….	…..
….	….	…..
Prerequisite_number	XXXXNNNN	PREREQUISITE

Note: Major_type is defined as an enumerared type with all known majors. XXXXNNNN is used to define a type with four alpha characters followed by four digits

Figure 1.3
An example of a database catalog for the database in Figure 1.2.

Main Characteristics of the Database Approach (cont'd.)



- **Data Abstraction:**
  - A **data model** is used to hide storage/implementation details and present the users with a conceptual view of the database.
  - Programs refer to the data model constructs rather than data storage details
- **Support of multiple views of the data:**
  - Each user may see a different view of the database, which describes **only** the data of interest to that user.
  - A view may be a **subset** of the database or it may contain **virtual data** that is derived from the database files but is not explicitly stored.

COURSE_PREREQUISITES

Course_name	Course_number	Prerequisites
Database	CS3380	CS3320
		MATH2410
Data Structures	CS3320	CS1310

Main Characteristics of the Database Approach (cont'd.)



- **Sharing of data and multi-user transaction processing:**
  - Allowing a set of **concurrent users** to retrieve from and to update the database.
  - *Concurrency control* within the DBMS guarantees that concurrent accesses (reads and updates) operate correctly and efficiently.
  - *Recovery* subsystem ensures each completed transaction has its effect permanently recorded in the database
  - **OLTP** (Online Transaction Processing) is a major part of database applications. This allows hundreds of concurrent transactions to execute per second.

Database Users



- **Users may be divided into**
  - Actors on the Scene: Those who actually use and control the database content, and those who design, develop and maintain database applications.
  - Workers Behind the Scene: Those who design and develop the DBMS software and related tools, and the computer systems operators but who are not actively interested in the database contents as part of their daily job.

Actors on the Scene



- **Database administrators:**
  - Responsible for authorizing access to the database, for coordinating and monitoring its use, acquiring software and hardware resources, controlling its use and monitoring efficiency of operations.
- **Database Designers:**
  - Responsible to define the content, the structure, the constraints, and functions or transactions against the database. They must communicate with the end-users and understand their needs.
- **End-users:**
  - They use the data for queries, reports and some of them update the database content. End-users can be categorized into:
    - Casual, Naïve or Parametric, Sophisticated, and Stand-alone users
- **System Analysts and Application Developers:**
  - System engineers who analyze, design, implement, debug, and deploy database applications

Workers behind the Scene



- **DBMS System Designers and Implementers:**
  - Design and implement the DBMS modules and interfaces as a software package.
- **Tool Developers:**
  - Design and implement tools for modeling and designing databases, performance monitoring, prototyping, test data generation, user interface creation, simulation etc.
- **Operators and Maintenance Personnel:**
  - They manage the actual running and maintenance of the database system hardware and software environment.

Advantages of Using the Database Approach



- **Controlling redundancy in data storage and in development and maintenance efforts.**
  - Sharing of data among multiple users.
- **Restricting unauthorized access to data. Only the DBA staff uses privileged commands and facilities.**
- **Providing persistent storage for program Objects**
  - E.g., Object-oriented DBMSs make program objects persistent– see Chapter 12.
- **Providing Storage Structures (e.g. indexes) for efficient Query Processing.**

Advantages of Using the Database Approach (cont'd.)



- **Providing optimization of queries for efficient processing.**
- **Providing backup and recovery services.**
- **Providing multiple interfaces to different classes of users.**
- **Representing complex relationships among data.**
- **Enforcing integrity constraints on the database.**
- **Permitting inferences and actions from the stored data using deductive and active rules and triggers.**

Additional Implications of Using the Database Approach



- **Potential for enforcing standards:**
  - This is very crucial for the success of database applications in large organizations. **Standards** refer to data item names, display formats, screens, report structures, meta-data (description of data), Web page layouts, etc.
- **Reduced application development time:**
  - Incremental time to add each new application is reduced.
- **Flexibility** to change data structures:
  - Database structure may evolve as new requirements are defined.
- **Availability of up-to-date information:**
  - As soon as one user's update is applied to the database, all other users can immediately see this update.
- **Economies of scale:**
  - Wasteful overlap of resources and personnel can be avoided by consolidating data and applications across departments.

Historical Development of Database Technology



- **Early Database Applications using Hierarchical and Network Models:**
  - Introduced in mid 1960s and dominated during the seventies.
  - Disadvantages:
    - Similar information would be kept many times
    - Intermixing of conceptual relationships with the physical storage and placement of records on disk (insufficient data abstraction)
    - Provide only Programming Language Interface (time-consuming to implement new queries and transactions)

- **Providing Data Abstraction and Application Flexibility**
- **with Relational Databases:**
  - Originally introduced in 1970, was heavily researched and experimented within IBM Research and several universities.
  - Relational DBMS Products emerged in the early 1980s.
  - Advantages:
    - Data abstraction: separate the physical storage of data from its conceptual representation
    - Provide a mathematical foundation for data representation and querying.
    - Introduce high-level query languages

Historical Development of Database Technology (cont'd.)



- **Object-Oriented Applications and the Need for More Complex Databases:**
  - Introduced in late 1980s and early 1990s to cater to the need of complex data processing in CAD and other applications.
    - Their use has not taken off much.
  - Many relational DBMSs have incorporated object database concepts, leading to a new category called *object-relational* DBMSs (ORDBMSs)
  - Pros and Cons:
    - Provide more general data structures.
    - Incorporated many of the useful object-oriented paradigms, such as abstract data types, encapsulation of operations, inheritance, and object identity.
    - However, the complexity of the model and the lack of an early standard contributed to their limited use.

Historical Development of Database Technology (cont'd.)



- **Data on the Web and E-commerce Applications:**
  - Web contains data in HTML (Hypertext markup language) with links among pages.
  - This has given rise to a new set of applications and E-commerce is using new standards like **XML** (eXtended Markup Language).
  - Script programming languages such as PHP and JavaScript allow generation of dynamic Web pages that are partially generated from a database.
    - Also allow database updates through Web pages

Extending Database Capabilities for New Applications



- **New functionality is being added to DBMSs in the following areas:**
  - **Scientific** Applications – Physics, Chemistry, Biology - Genetics
  - Earth and Atmospheric Sciences and Astronomy
  - **Image** Storage and Management
  - **Audio** and **Video** Data Management
  - **Data Mining** – a very major area for future development using new technologies
  - **Spatial** Data Management and Location Based Services
  - **Time Series** and **Historical Data** Management
- **The above gives rise to new research and development in incorporating**
  - **More complex data structures**
  - **New data types**
  - **New operations and query language constructs**
  - **New storage and indexing schemes**

Extending Database Capabilities (cont'd.)



- **Emergence of Big Data Technologies and NOSQL databases**
  - New data storage, management and analysis technology was necessary to deal with the onslaught of data in **petabytes** a day (10**15 bytes or 1000 terabytes) in some applications – this started being commonly called as "**Big Data**".
  - **Hadoop** (which originated from Yahoo) and **MapReduce Programming** approach to distributed data processing (which originated from Google) as well as the **Google file system** have given rise to Big Data technologies (Chapter 25). Further enhancements are taking place in the form of **Spark** based technology.
  - **NOSQL** (Not Only SQL - where SQL is the *de facto* standard language for relational DBMSs) systems have been designed for rapid search and retrieval from documents, processing of huge graphs occurring on social networks, and other forms of unstructured data with flexible models of transaction processing .

When not to use a DBMS



- **Main inhibitors (costs) of using a DBMS:**
  - High initial investment and possible need for additional hardware.
  - Overhead for providing generality, security, concurrency control, recovery, and integrity functions.
- **When a DBMS may be unnecessary:**
  - If the database and applications are simple, well defined, and not expected to change.
  - No multiple-user access to data.
- **When a DBMS may be infeasible:**
  - In embedded systems where a general purpose DBMS may not fit in available storage
  - Stringent, real-time requirements may not be met because of DBMS overhead
  - Complex data modeling (e.g., in complex genome and protein databases)
  - Special operations not supported by the DBMS (e.g., GIS and location based services).